Flower Image Classification using CNN and Transfer Learning (EfficientNetB0 with Fine-Tuning)

Student Name: Mounika Balina
Student ID: 22097217
GitHub Link: https://github.com/balina1234/Individual-Assignment-Machine-Learning-Tutorial.git
Dataset Link: https://www.kaggle.com/datasets/nadyana/flowers

Introduction

Image classification is a key task in computer vision, where images are automatically assigned labels based on their content. It is widely used in areas such as healthcare, autonomous driving, and retail. Traditional machine learning approaches relied heavily on handcrafted features; however, these methods often struggled to generalise across complex and diverse image datasets.

With the advancement of deep learning, Convolutional Neural Networks (CNNs) have become the dominant approach for image classification. They automatically learn hierarchical features from raw pixel data, capturing both low-level patterns (e.g., edges) and high-level semantic information. However, training CNNs from scratch requires large datasets and substantial computational resources.

To address these challenges, transfer learning has become a widely adopted technique. It leverages pre-trained models that have already learned rich feature representations from large-scale datasets such as ImageNet. In this tutorial, the EfficientNetB0 architecture is used due to its efficient scaling and strong performance across various image classification benchmarks.

This tutorial focuses on three key stages. First, a baseline CNN model is developed from scratch to establish a performance benchmark. Second, transfer learning is applied using EfficientNetB0 with frozen layers to utilise pre-trained knowledge. Finally, fine-tuning is performed by unfreezing selected layers of the pre-trained model, allowing it to adapt more effectively to the target dataset.

The primary objective of this tutorial is to demonstrate how transfer learning and fine-tuning can significantly enhance classification accuracy and generalisation compared to traditional CNN models. Additionally, the tutorial provides a complete end-to-end pipeline, including data cleaning, exploratory data analysis, preprocessing, model training, evaluation, and visualisation.

Dataset and Preprocessing

Dataset Description

The dataset used in this study consists of flower images organised into separate folders where each folder represents a distinct class label. The classes included are *rose, sunflower, tulip, daisy, bellflower, dandelion* and *lotus*. Each category contains **1600 images**, resulting in a **balanced dataset** with equal representation across all classes.

Data Cleaning

A custom data cleaning function was implemented to ensure dataset quality and robustness. The cleaning process involved:

- Removing files that do not match supported image formats (e.g., .jpg, .jpeg, .png)
- Detecting and deleting corrupted or unreadable images using image verification techniques

This step is critical because invalid or corrupted images can lead to:

- Training interruptions or runtime errors
- Incorrect feature learning
- Reduced model performance

By ensuring that only valid images are included, the dataset becomes more reliable for training deep learning models.

Data Loading

The dataset is loaded using TensorFlow's image_dataset_from_directory() function, which simplifies image preprocessing and batching. The following configurations were used:

- **Training set:** 80% of the dataset
- **Validation set:** 20% of the dataset
- **Image size:** 224 × 224 pixels
- **Batch size:** 32
- **Random seed:** 42 (for reproducibility)

Resizing all images to a fixed dimension ensures compatibility with deep learning models, particularly EfficientNetB0, which requires a consistent input shape. Batching improves computational efficiency and speeds up training.

Exploratory Data Analysis (EDA)

Exploratory Data Analysis (EDA) was performed to better understand the dataset before training the models.

Sample Image Visualization

A subset of images from different classes was visualised (as shown below), providing insight into:

Variations in colour, texture, shape and

Differences in background environments

Class Distribution Analysis

The dataset contains an equal number of images - 1600. This uniform distribution indicates that the dataset is **perfectly balanced**, which eliminates the need for techniques such as class weighting or resampling

Normalisation
Pixel values in images originally range from **0 to 255**.
These values were scaled to the range: [0, 255]-> [0, 1]
This normalisation step is essential because:

- It ensures numerical stability during training
- It prevents large gradients that can slow down learning
- It accelerates convergence of the model
- It improves overall model performance

For the baseline CNN, normalisation is applied directly using a rescaling layer. For EfficientNetB0, a specialised preprocessing function (preprocess_input) is used to match the input distribution expected by the pre-trained model.

Baseline CNN Model

Model Architecture
The baseline model is a Convolutional Neural Network (CNN) designed to perform multi-class image classification. It consists of three convolutional blocks with increasing filter sizes (32, 64, and 128), enabling the model to progressively learn more complex spatial features. Each convolutional layer is followed by a max-pooling layer, which reduces the spatial dimensions and helps retain the most important features.

After feature extraction, the output is flattened and passed through a fully connected dense layer with 128 neurons. A dropout layer with a rate of 0.5 is applied to reduce overfitting by randomly disabling neurons during training. Finally, a softmax output layer is used to produce probability distributions across the seven flower classes.

Training Configuration
The model is trained using the following settings: **Optimizer:** Adam, **Loss Function:** Sparse Categorical Crossentropy, **Evaluation Metric:** Accuracy and **Epochs:** 10
Adam optimizer is chosen due to its adaptive learning capability, which helps achieve faster convergence during training.

Results and Performance Analysis

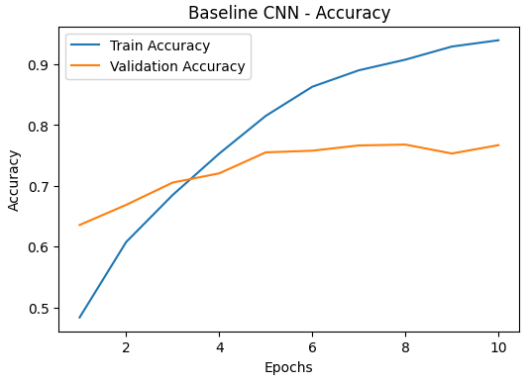
Accuracy and Loss Trends
From the training and validation curves:

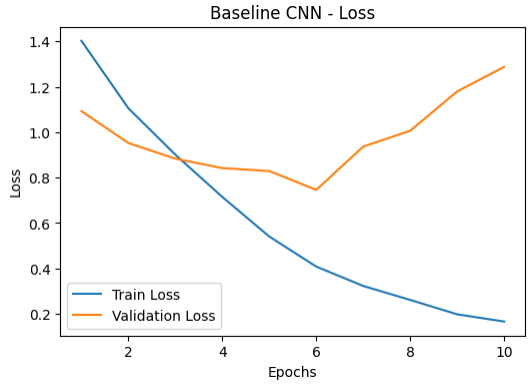
- Training accuracy increases steadily from 48% to 94%, indicating that the model is learning patterns effectively from the training data.
- Validation accuracy improves initially but stabilises around 75–77%, showing limited generalisation.

Similarly, loss curves show:

- Training loss decreases consistently, reaching a very low value (0.17)

- Validation loss initially decreases but then increases after epoch 6

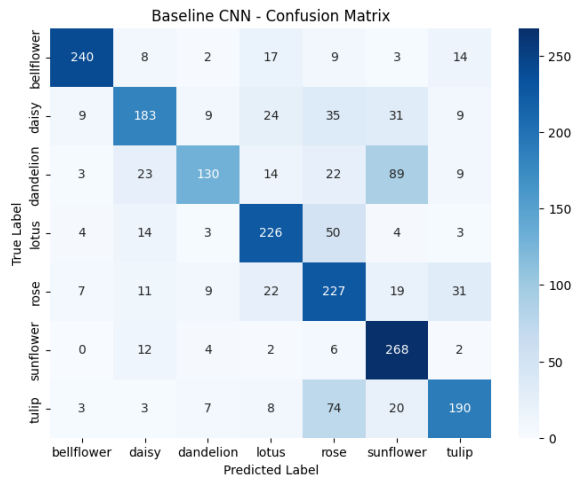




Confusion Matrix Insights

- The confusion matrix further highlights misclassification patterns:
- Significant confusion between dandelion and sunflower
- Misclassification between rose and tulip
- Some overlap between daisy and other flower types

These errors occur because the baseline CNN lacks the ability to extract highly discriminative features required for fine-grained classification.



Observations

From the overall evaluation:

- The model achieves **moderate accuracy - 69%**, indicating baseline learning capability
- Clear signs of **overfitting** are observed due to divergence in training and validation metrics
- Performance varies significantly across classes
- The model struggles with **complex feature extraction and class similarity**

Transfer Learning with EfficientNetB0

Concept of Transfer Learning

Transfer learning is a powerful deep learning technique where a model trained on a large-scale dataset (such as ImageNet) is reused for a new, related task. Instead of learning features from scratch, the model leverages previously learned representations such as edges, textures, shapes, and object structures.
This approach offers several advantages: Reduces training time significantly, Requires less labelled data Improves model performance & generalisation and Avoids overfitting in smaller datasets
In this tutorial, transfer learning is applied using EfficientNetB0 to improve classification accuracy compared to the baseline CNN.

EfficientNetB0 Model

EfficientNetB0 is a state-of-the-art convolutional neural network that introduces **compound scaling**, which uniformly scales: Network depth, Width (number of channels) and Input resolution

This balanced scaling allows EfficientNet to achieve high accuracy while maintaining computational efficiency. It is particularly suitable for this task because it provides strong feature extraction capabilities with relatively low complexity.

Implementation

The transfer learning pipeline involves the following steps:

1. Load the pre-trained EfficientNetB0 model with ImageNet weights
2. Remove the original classification head (include_top=False)
3. Freeze all base model layers to preserve learned features
4. Add a custom classification head: Global Average Pooling, Batch Normalisation, Dense layer (128 units), Dropout (0.5) and Softmax output layer
5. Apply EfficientNet-specific preprocessing (preprocess_input)
6. Train only the newly added layers

Data Augmentation

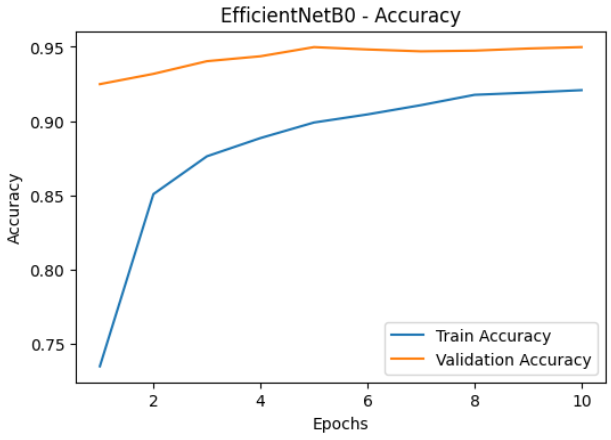
To further improve generalisation, data augmentation techniques are applied: Random horizontal flipping, Rotation and Zoom.

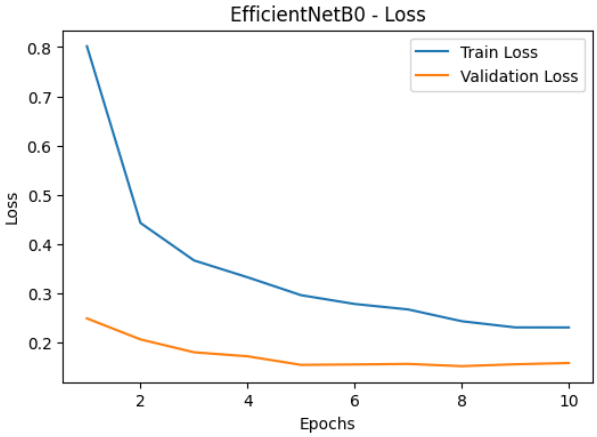
These transformations simulate real-world variations and help the model become more robust to unseen data.

Results and Performance Analysis

Accuracy and Loss Trends

- Training accuracy increases from **73% to 92%**
- Validation accuracy remains consistently high at **94–95%**
- Validation loss is lower than training loss throughout training





This indicates:

- Strong generalisation
- No overfitting
- Effective feature extraction from the pre-trained model

Classification Report Analysis

The Accuracy for EfficientNetB0 is 95%

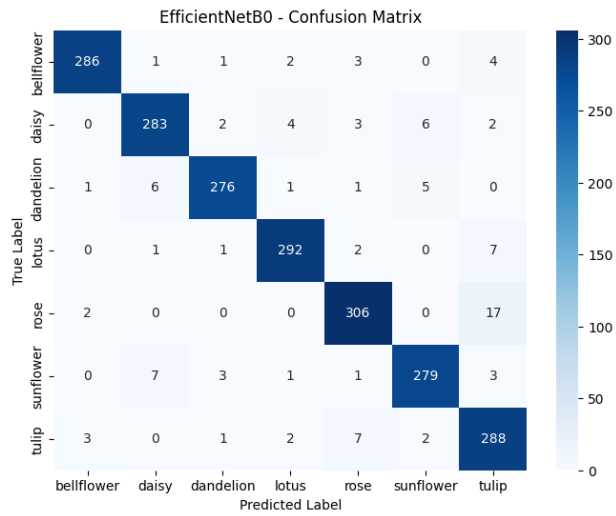
Class-wise performance shows:

- **Bellflower, Lotus, Dandelion:** Near-perfect classification (F1 = 0.96–0.97)
- **Daisy and Sunflower:** Strong and consistent predictions
- **Tulip (F1 = 0.92):** Slightly lower performance, indicating minor confusion with similar classes
- **Rose (F1 = 0.94):** Well-classified with balanced precision and recall

Overall, the model demonstrates **high consistency across all classes**, unlike the baseline CNN.

Confusion Matrix Insights

The confusion matrix shows: Very strong diagonal dominance (correct predictions), Minimal misclassification between classes and Slight confusion between visually similar classes (e.g., tulip and rose). Compared to the baseline CNN, errors are significantly reduced.



Key Observations

- Accuracy improves dramatically from **69% → 95%**
- Model generalises well without overfitting
- Learned features from ImageNet significantly enhance classification
- Training is more efficient and stable

This confirms that **transfer learning is far superior to training from scratch** for this task.

Fine-Tuning EfficientNet

Concept of Fine-Tuning

Fine-tuning is an extension of transfer learning where some layers of the pre-trained model are unfrozen and retrained on the new dataset.

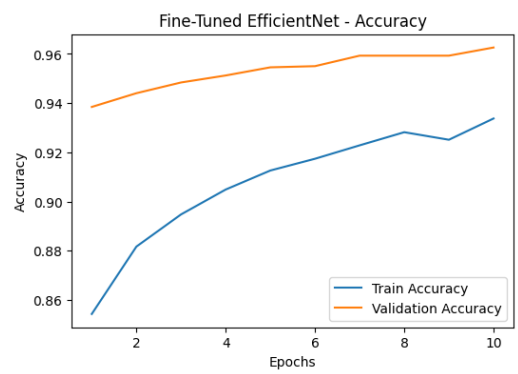
Instead of using fixed features:

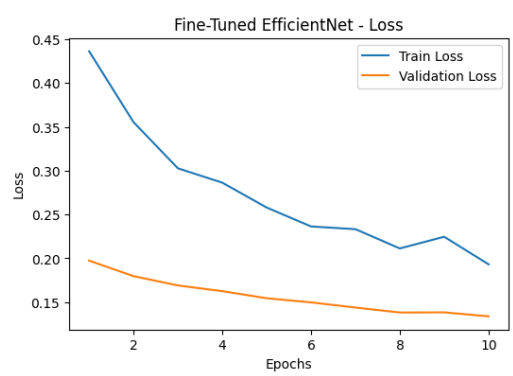
- The model adapts higher-level features to the specific dataset
- Improves task-specific representation

Implementation

- The last **50 layers of EfficientNetB0 are unfrozen**
- A **low learning rate (1e-5)** is used

This ensures: Small, stable updates to weights, Preservation of useful pre-trained knowledge and Reduced risk of overfitting





Results and Performance Analysis

Accuracy and Loss Trends

- Training accuracy improves further to **93–94%**
- Validation accuracy increases to **96%**

- Both training and validation loss decrease steadily

This indicates: Improved model learning , Better adaptation to dataset-specific features and No overfitting

Classification Report Analysis

Accuracy: 96%

Class-wise insights:

- **All classes achieve very high F1-scores (≥ 0.95)**
- **Bellflower (F1 = 0.98):** Highest performance
- **Sunflower & Daisy:** Near-perfect classification
- **Tulip and Rose:** Slight improvement over previous model

Performance is now **uniform across all classes** indicating strong feature learning.

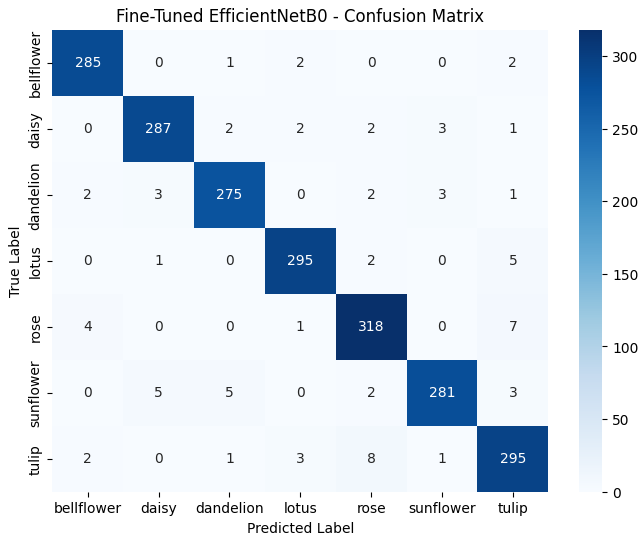
Confusion Matrix Insights

- Nearly perfect diagonal dominance
- Very few misclassifications
- Minimal confusion between similar flower types

This demonstrates that fine-tuning allows the model to better distinguish subtle differences between classes.

Key Observations

- Accuracy improves from **95% → 96%**
- Model achieves **state-of-the-art performance** on this dataset
- Feature representations become more refined
- Class-wise performance becomes more balanced



Model Comparison and Discussion

Quantitative Model Comparison

The performance of the three models is summarised below:

Model	Accuracy	Precision	Recall	F1-Score
Baseline CNN	0.693	0.715	0.693	0.691
EfficientNet (Frozen)	0.952	0.952	0.952	0.952
Fine-Tuned EfficientNet	0.964	0.964	0.964	0.964

The results clearly demonstrate a significant improvement in performance when moving from a baseline CNN to transfer learning approaches.

- The **baseline CNN** achieves moderate accuracy (69%) indicating limited feature extraction capability.
- The **frozen EfficientNet model** shows a dramatic improvement (95%) highlighting the effectiveness of pre-trained features.
- The **fine-tuned EfficientNet model** achieves the highest performance (96%) demonstrating the benefit of adapting learned features to the target dataset.

Prediction Visualisation Analysis

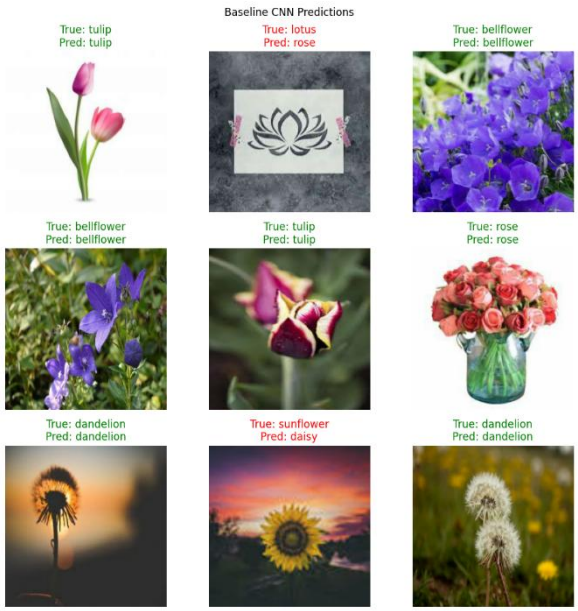
Baseline Model Predictions

Figure 1 shows predictions from the baseline CNN. While some classifications are correct, several misclassifications occur, particularly between visually similar classes such as sunflower vs daisy and lotus vs rose. This indicates limited feature extraction, as the model struggles to distinguish subtle differences in colour, texture and shape.

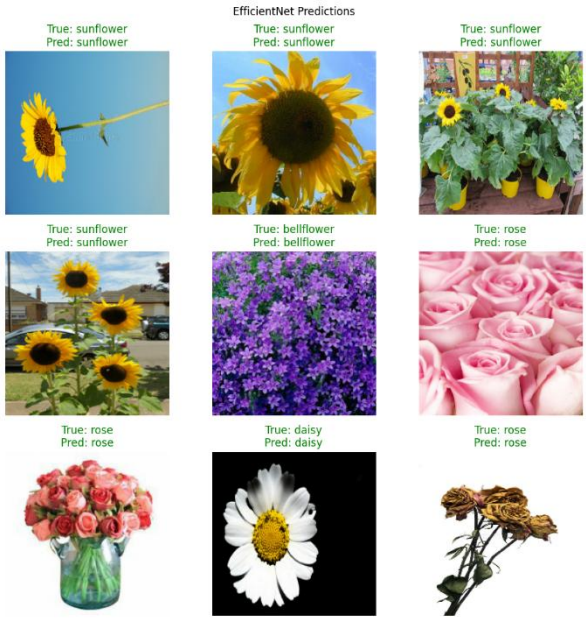
EfficientNet Predictions (Frozen Model)

Figure 2 presents predictions from the transfer learning model. Compared to the baseline, predictions are more accurate and consistent, with most images correctly classified. This improvement highlights the effectiveness of pre-trained features in capturing complex visual patterns and improving model performance.

Fig(1) : Baseline CNN Model Predictions



Fig(2) : EfficientNet Model Predictions



Conclusion

This Tutorial demonstrates the effectiveness of transfer learning for image classification. The baseline CNN achieved moderate performance due to limited feature learning whereas the pre-trained EfficientNetB0 significantly improved accuracy and generalisation by leveraging knowledge from large-scale datasets.

Freezing the base model provided strong initial results, while fine-tuning further enhanced performance by adapting features to the flower dataset, achieving the highest accuracy with minimal misclassification. The results highlight that **feature quality is more important than model size** and transfer learning consistently outperforms models trained from scratch. It is particularly effective when data and computational resources are limited.

Overall, fine-tuned transfer learning offers a powerful, efficient and scalable approach for image classification and can be applied to a wide range of real-world computer vision tasks.

References

- **Tan, M. and Le, Q. (2019)** *EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks*. Available at: https://arxiv.org/abs/1905.11946
- **Krizhevsky, A., Sutskever, I. and Hinton, G.E. (2012)** *ImageNet Classification with Deep Convolutional Neural Networks*. Available at: https://papers.nips.cc/paper_files/paper/2012
- **Pan, S.J. and Yang, Q. (2010)** *A Survey on Transfer Learning*. IEEE Transactions on Knowledge and Data Engineering. Available at: https://ieeexplore.ieee.org/document/5288526
- **Chollet, F. (2017)** *Deep Learning with Python*. Manning Publications.
- **TensorFlow (2024)** *Transfer learning and fine-tuning tutorial*. Available at: https://www.tensorflow.org/tutorials/images/transfer_learning
- **Kaggle (2024)** *Flowers Dataset*. Available at: https://www.kaggle.com/datasets/nadyana/flowers